***BRACT's***

**Vishwakarma Institute of Technology, Pune**

**Practical Implementation Sheet**

Department: CSE-AI Semester: 5 Academic Year: 2026-27

Division: E Batch: 2 Practical No: 10

Roll no: 53 PRN No: 12414903 Name of Student: **Sara Ansingkar**

**Subject Name : Deep Learning**

## Problem Statement :

Design and implement deep learning application for audio classification that incorporates Speech to Text (STT) and Text to Speech (TTS) using appropriate deep learning architecture and evaluate its suitable performance using suitable metrics.

### ALGORITHM:

1. Start the application.

2. Import libraries required for audio processing, deep learning, speech recognition, text-to-speech, and evaluation.

3. Download the RAVDESS dataset using KaggleHub.

4. Preprocess the audio data by loading WAV files, converting them to a fixed sampling rate, and padding or truncating recordings to a fixed duration.

5. Extract MFCC features from each audio recording to represent its spectral characteristics.

6. Label the audio samples according to their corresponding emotions: neutral, calm, happy, sad, angry, fearful, disgust, and surprised.

7. Split the dataset into training, validation, and testing sets, ensuring that actors do not overlap between the sets.

8. Build the CNN model using convolutional layers, batch normalization, pooling, dropout, and a softmax output layer.

9. Train the CNN model using the training dataset and monitor validation accuracy and loss. Apply early stopping to reduce overfitting.

10. Evaluate the trained model using accuracy, precision, recall, F1-score, and a confusion matrix.

11. Accept audio input from the user through file upload or microphone recording.

12. Perform Speech-to-Text (STT) using the pretrained Whisper model to convert the spoken audio into text.

13. Predict the emotion by extracting MFCC features from the input audio and passing them to the trained CNN model.

14. Determine the predicted emotion and confidence score from the softmax output.

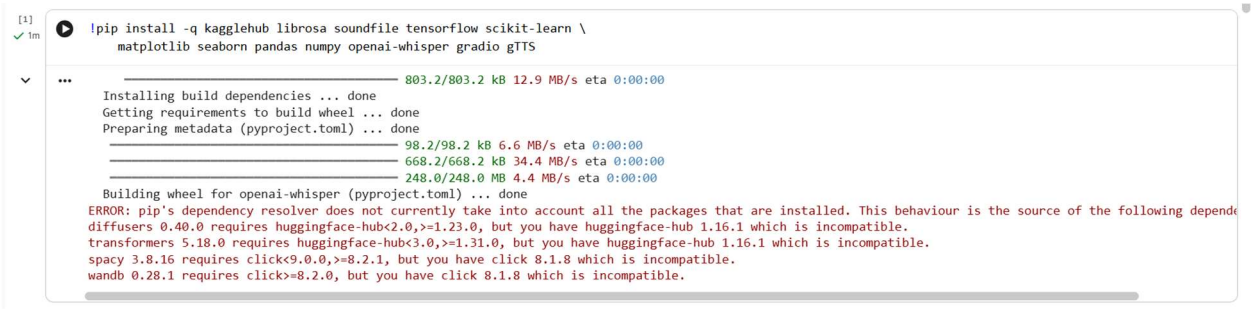
15. Generate a textual response containing the recognized speech and predicted emotion.

16. Perform Text-to-Speech (TTS) using gTTS to convert the response into spoken audio.

17. Display the results in the Gradio interface, including the transcript, predicted emotion, confidence score, and generated audio.

18. Stop the application.

## Code and Output:



[2] ✓ 4s

```
%pip install --upgrade "huggingface-hub>=1.31.0,<2.0.0" "click>=8.2.1,<9.0.0"
```

```
Collecting huggingface-hub<2.0.0,>=1.31.0
  Downloading huggingface_hub-1.33.0-py3-none-any.whl.metadata (16 kB)
Collecting click<9.0.0,>=8.2.1
  Downloading click-8.5.0-py3-none-any.whl.metadata (2.6 kB)
Requirement already satisfied: filelock>=3.10.0 in /usr/local/lib/python3.13/dist-packages (from huggingface-hub<2.0.0,>=1.31.0) (4.0.7)
Requirement already satisfied: fsspec>=2023.5.0 in /usr/local/lib/python3.13/dist-packages (from huggingface-hub<2.0.0,>=1.31.0) (2025.12.0)
Requirement already satisfied: hf-xet<2.0.0,>=1.6.0 in /usr/local/lib/python3.13/dist-packages (from huggingface-hub<2.0.0,>=1.31.0) (1.6.0)
Requirement already satisfied: httpx<1,>=0.23.0 in /usr/local/lib/python3.13/dist-packages (from huggingface-hub<2.0.0,>=1.31.0) (0.28.1)
Requirement already satisfied: packaging>=20.9 in /usr/local/lib/python3.13/dist-packages (from huggingface-hub<2.0.0,>=1.31.0) (26.3)
Requirement already satisfied: pyyaml>=5.1 in /usr/local/lib/python3.13/dist-packages (from huggingface-hub<2.0.0,>=1.31.0) (6.0.3)
Requirement already satisfied: tqdm>=4.42.1 in /usr/local/lib/python3.13/dist-packages (from huggingface-hub<2.0.0,>=1.31.0) (4.67.3)
Requirement already satisfied: typing-extensions>=4.1.0 in /usr/local/lib/python3.13/dist-packages (from huggingface-hub<2.0.0,>=1.31.0) (4.16.0)
Requirement already satisfied: anyio in /usr/local/lib/python3.13/dist-packages (from httpx<1,>=0.23.0->huggingface-hub<2.0.0,>=1.31.0) (4.15.1)
Requirement already satisfied: certifi in /usr/local/lib/python3.13/dist-packages (from httpx<1,>=0.23.0->huggingface-hub<2.0.0,>=1.31.0) (2026.7.22)
Requirement already satisfied: httpcore==1.* in /usr/local/lib/python3.13/dist-packages (from httpx<1,>=0.23.0->huggingface-hub<2.0.0,>=1.31.0) (1.0.9)
Requirement already satisfied: idna in /usr/local/lib/python3.13/dist-packages (from httpx<1,>=0.23.0->huggingface-hub<2.0.0,>=1.31.0) (3.20)
Requirement already satisfied: h11>=0.16 in /usr/local/lib/python3.13/dist-packages (from httpcore==1.*->httpx<1,>=0.23.0->huggingface-hub<2.0.0,>=1.31.0) (0.1
Downloading huggingface_hub-1.33.0-py3-none-any.whl (846 kB)
   ━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━ 846.4/846.4 kB 12.8 MB/s eta 0:00:00
Downloading click-8.5.0-py3-none-any.whl (125 kB)
   ━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━ 125.2/125.2 kB 8.5 MB/s eta 0:00:00
```

[4] ✓ 4s

```
import kagglehub
import os

path = kagglehub.dataset_download(
    "uwrfkaggler/ravdess-emotional-speech-audio"
)

print("Path to dataset files:", path)

# Inspect the downloaded dataset
for root, dirs, files in os.walk(path):
    if files:
        print("Folder:", root)
        print("Number of files:", len(files))
        print("Sample files:", files[:5])
        break
```

```
Using Colab cache for faster access to the 'ravdess-emotional-speech-audio' dataset.
Path to dataset files: /kaggle/input/ravdess-emotional-speech-audio
Folder: /kaggle/input/ravdess-emotional-speech-audio/Actor_02
Number of files: 60
Sample files: ['03-01-08-01-01-01-02.wav', '03-01-01-01-01-01-02.wav', '03-01-07-02-01-02-02.wav', '03-01-07-01-01-02-02.wav', '03-01-01-01-02-01-02.wav']
```

5] 8s

```
import os
import glob
import numpy as np
import pandas as pd
import librosa
import matplotlib.pyplot as plt
import seaborn as sns
import tensorflow as tf

from sklearn.model_selection import GroupShuffleSplit
from sklearn.preprocessing import LabelEncoder
from sklearn.metrics import (
    accuracy_score,
    classification_report,
    confusion_matrix
)

from tensorflow.keras.models import Sequential
from tensorflow.keras.layers import (
    Input, Conv2D, MaxPooling2D, BatchNormalization,
    Flatten, Dense, Dropout
)
from tensorflow.keras.callbacks import EarlyStopping
```

```
SEED = 42
np.random.seed(SEED)
tf.random.set_seed(SEED)

SAMPLE_RATE = 22050
DURATION = 4
N_MFCC = 40
MAX_LEN = 174

print("TensorFlow version:", tf.__version__)
print("GPU available:", tf.config.list_physical_devices("GPU"))
```

```
TensorFlow version: 2.21.0
GPU available: []
```

[6] ✓ 17s

```
audio_files = glob.glob(
    os.path.join(path, "**", "*.wav"),
    recursive=True
)

print("Total WAV files found:", len(audio_files))

# Emotion ID to emotion name
emotion_map = {
    "01": "neutral",
    "02": "calm",
    "03": "happy",
    "04": "sad",
    "05": "angry",
    "06": "fearful",
    "07": "disgust",
    "08": "surprised"
}

records = []
```

```
for file in audio_files:
    filename = os.path.basename(file)
    parts = filename.replace(".wav", "").split("-")

    if len(parts) != 7:
        continue

    emotion_id = parts[2]
    statement_id = parts[4]
    actor_id = parts[6]

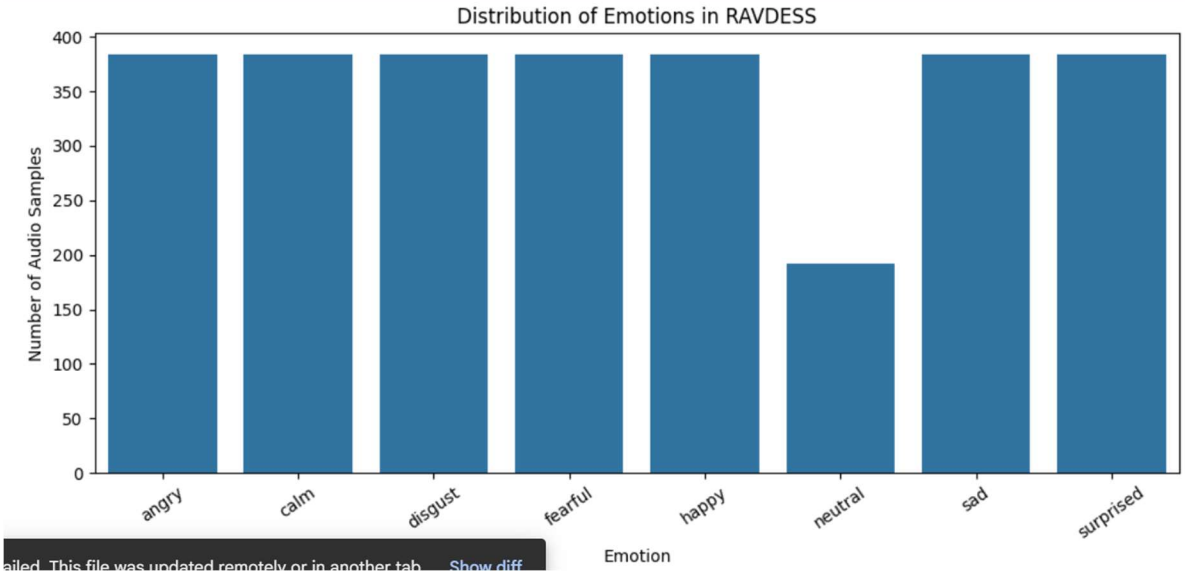
    if emotion_id in emotion_map:
        records.append({
            "file": file,
            "emotion": emotion_map[emotion_id],
            "emotion_id": emotion_id,
            "statement_id": statement_id,
            "actor_id": int(actor_id)
        })

df = pd.DataFrame(records)
```

```
Emotion distribution:
emotion
surprised    384
disgust      384
fearful      384
sad          384
happy        384
calm         384
angry        384
neutral      192
Name: count, dtype: int64

Sample records:
```

| | file | emotion | emotion_id | statement_id | actor_id |
|---|---|---|---|---|---|
| 0 | /kaggle/input/ravdess-emotional-speech-audio/A... | surprised | 08 | 01 | 2 |
| 1 | /kaggle/input/ravdess-emotional-speech-audio/A... | neutral | 01 | 01 | 2 |
| 2 | /kaggle/input/ravdess-emotional-speech-audio/A... | disgust | 07 | 01 | 2 |
| 3 | /kaggle/input/ravdess-emotional-speech-audio/A... | disgust | 07 | 01 | 2 |
| 4 | /kaggle/input/ravdess-emotional-speech-audio/A... | neutral | 01 | 02 | 2 |



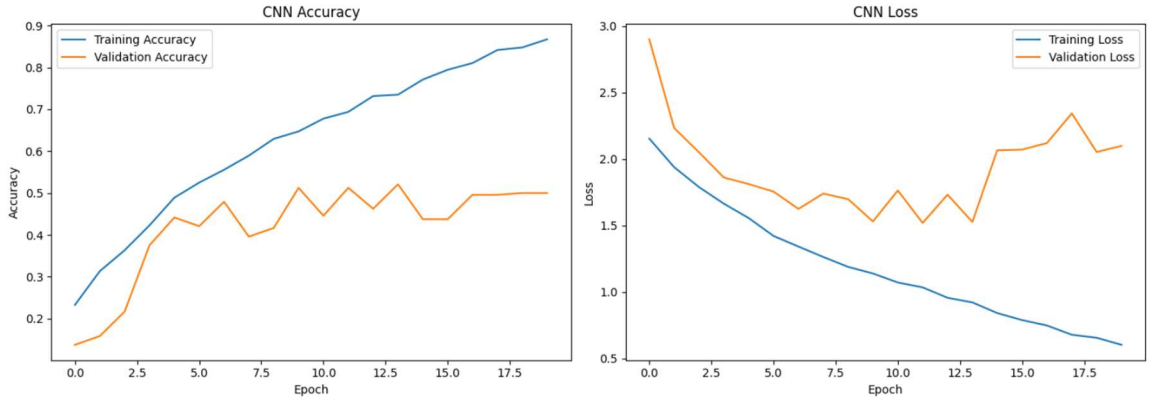

ailed. This file was updated remotely or in another tab. Show diff

```
Feature tensor shape: (2880, 40, 174, 1)
Labels shape: (2880,)
```

```
Training samples: 1800
Validation samples: 480
Testing samples: 600
Training actors: [np.int64(4), np.int64(5), np.int64(6), np.int64(8), np.int64(10), np.int64(11), np.int64(13), np.int64(14), np.int64(16), np.int64(18), np.int
Validation actors: [np.int64(2), np.int64(3), np.int64(7), np.int64(15)]
Testing actors: [np.int64(1), np.int64(9), np.int64(12), np.int64(17), np.int64(19)]
```

| Layer (type) | Output Shape | Param # |
|---|---|---|
| conv2d (Conv2D) | (None, 40, 174, 32) | 320 |
| batch_normalization (BatchNormalization) | (None, 40, 174, 32) | 128 |
| max_pooling2d (MaxPooling2D) | (None, 20, 87, 32) | 0 |
| conv2d_1 (Conv2D) | (None, 20, 87, 64) | 18,496 |
| batch_normalization_1 (BatchNormalization) | (None, 20, 87, 64) | 256 |
| max_pooling2d_1 (MaxPooling2D) | (None, 10, 43, 64) | 0 |
| conv2d_2 (Conv2D) | (None, 10, 43, 128) | 73,856 |
| batch_normalization_2 (BatchNormalization) | (None, 10, 43, 128) | 512 |
| max_pooling2d_2 (MaxPooling2D) | (None, 5, 21, 128) | 0 |
| flatten (Flatten) | (None, 13440) | 0 |
| dense (Dense) | (None, 128) | 1,720,448 |
| dropout (Dropout) | (None, 128) | 0 |
| dense_1 (Dense) | (None, 8) | 1,032 |

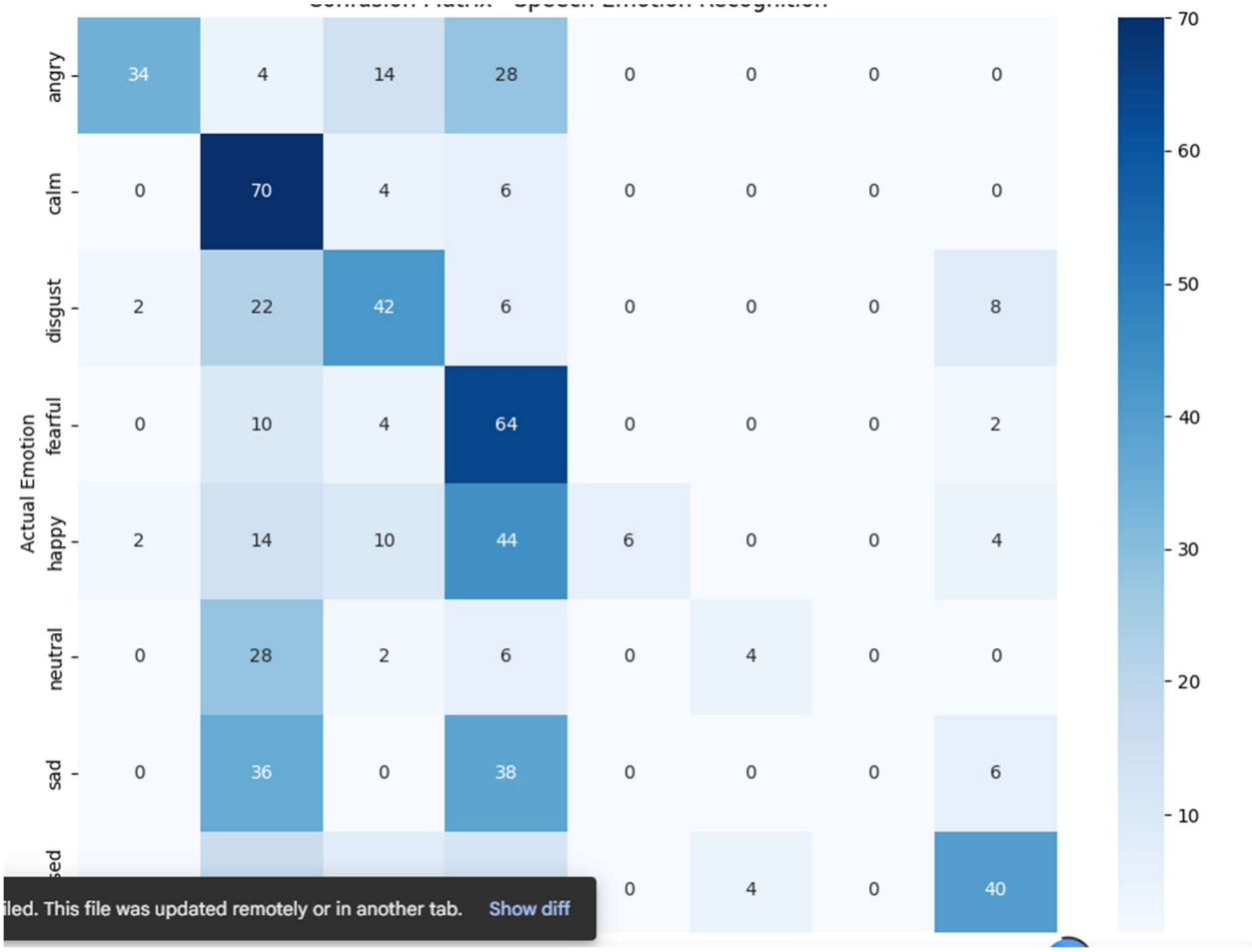
**Total params:** 1,815,048 (6.92 MB)
**Trainable params:** 1,814,600 (6.92 MB)
**Non-trainable params:** 448 (1.75 KB)



Test Accuracy: 43.33%

Classification Report:

| | precision | recall | f1-score | support |
|---|---|---|---|---|
| angry | 0.89 | 0.42 | 0.58 | 80 |
| calm | 0.35 | 0.88 | 0.50 | 80 |
| disgust | 0.50 | 0.53 | 0.51 | 80 |
| fearful | 0.31 | 0.80 | 0.45 | 80 |
| happy | 1.00 | 0.07 | 0.14 | 80 |
| neutral | 0.50 | 0.10 | 0.17 | 40 |
| sad | 0.00 | 0.00 | 0.00 | 80 |
| surprised | 0.67 | 0.50 | 0.57 | 80 |
| accuracy | | | 0.43 | 600 |
| macro avg | 0.53 | 0.41 | 0.36 | 600 |
| weighted avg | 0.53 | 0.43 | 0.38 | 600 |




iled. This file was updated remotely or in another tab. Show diff

```
Reference: Dogs are sitting by the door
Predicted: Dogs are sitting by the door.
--------------------------------------------------
Reference: Kids are talking by the door
Predicted: Kids are talking by the door.
--------------------------------------------------
Reference: Kids are talking by the door
Predicted: Kids are talking by the door.
--------------------------------------------------
Reference: Kids are talking by the door
Predicted: Kids are talking by the door.
--------------------------------------------------
Reference: Dogs are sitting by the door
Predicted: Dogs are sitting by the door.
--------------------------------------------------
Reference: Dogs are sitting by the door
Predicted: Dogs are sitting by the door.
--------------------------------------------------
Reference: Dogs are sitting by the door
Predicted: The dogs are sitting by the door!
--------------------------------------------------
Reference: Dogs are sitting by the door
Predicted: Dogs are sitting by the door.
--------------------------------------------------
Reference: Kids are talking by the door
Predicted: Kids are talking by the door!
--------------------------------------------------
Reference: Kids are talking by the door
Predicted: Kids are talking by the door.
--------------------------------------------------
Word Error Rate (WER): 20.00%
```

